

02 — ELT Patterns with PostgreSQL

Table of Contents

- 1. [Learning Objectives](#)
- 2. [ETL vs ELT — The Fundamental Difference](#)
- 3. [Why PostgreSQL is a Strong ELT Engine](#)
- 4. [In-Database Transformation Patterns](#)
- 5. [Staging Layer Design](#)
- 6. [Transform Layer Patterns](#)
- 7. [Orchestrating ELT with SQL](#)
- 8. [Common Mistakes](#)
- 9. [Best Practices](#)
- 10. [Performance Considerations](#)
- 11. [Interview Questions & Answers](#)
- 12. [Exercises & Solutions](#)
- 13. [Real-World Scenarios](#)
- 14. [Cross-References](#)

Learning Objectives

- Articulate the conceptual and practical difference between ETL and ELT
- Design a multi-layer ELT architecture (raw → staging → mart) inside PostgreSQL
- Write efficient in-DB transformation SQL using CTEs, window functions, and set operations
- Implement idempotent ELT transforms
- Know when ELT is superior to ETL and when it is not

ETL vs ELT — The Fundamental Difference

ETL (Extract → Transform → Load)

Source —▶ [Transform outside DB] —▶ Target DB
Python / Spark / Glue
(data leaves the DB)

ELT (Extract → Load → Transform)

Source —▶ Target DB (raw zone) —▶ [Transform inside DB] —▶ Mart
(load first, SQL / stored procs
transform later) (data stays in DB)

Dimension	ETL	ELT
Transform location	External tool	Inside the database
Scalability	Limited by ETL tool	Scales with DB compute
Flexibility	Schema defined upfront	Raw data stored first
Latency	Higher (more hops)	Lower (one load step)

Debugging	Easier (intermediate files)	SQL explains plan
Best for	Legacy DWs, compliance	Modern cloud DW, PostgreSQL

Why PostgreSQL is a Strong ELT Engine

PostgreSQL ships with capabilities that rival expensive ETL tools:

- **Window functions** — complex analytical transforms in pure SQL
- **CTEs (WITH clauses)** — multi-step pipelines in one statement
- **JSONB operators** — transform semi-structured data inline
- **Regular expressions** — `regexp_replace` , `regexp_matches`
- **plpgsql / plpython3u** — custom transform logic
- **Materialized Views** — persist transform results, refresh on demand
- **Table Partitioning** — parallelize transforms by partition
- **FDW (Foreign Data Wrappers)** — pull from remote sources in SQL
- **COPY FROM PROGRAM** — integrate shell scripts inline

In-Database Transformation Patterns

Pattern 1: CTE Pipeline Transform

```
-- Multi-step transform expressed as a single SQL statement
WITH
-- Step 1: Parse and clean raw data
raw_cleaned AS (
    SELECT
        (raw_payload->>'order_id')::BIGINT                AS order_id,
        TRIM(LOWER(raw_payload->>'email'))                AS email,
        (raw_payload->>'amount')::NUMERIC(12,2)           AS amount,
        (raw_payload->>'created_at')::TIMESTAMPTZ         AS created_at,
        UPPER(COALESCE(raw_payload->>'country', 'US'))    AS country_code
    FROM raw_orders
    WHERE loaded_at >= CURRENT_DATE - INTERVAL '1 day'
        AND raw_payload->>'order_id' IS NOT NULL
),
-- Step 2: Deduplicate (keep latest per order_id)
deduped AS (
    SELECT DISTINCT ON (order_id) *
    FROM raw_cleaned
    ORDER BY order_id, created_at DESC
),
-- Step 3: Enrich with customer dimension
enriched AS (
    SELECT d.*,
        c.customer_id,
        c.segment,
        CASE
            WHEN d.amount >= 1000 THEN 'high'
            WHEN d.amount >= 100  THEN 'medium'
```

```

        ELSE                                'low'
    END AS order_tier
FROM deduped d
LEFT JOIN customers c ON c.email = d.email
)
-- Step 4: Upsert into target mart
INSERT INTO mart_orders (order_id, customer_id, email, amount, country_code,
order_tier, created_at)
SELECT order_id, customer_id, email, amount, country_code, order_tier, created_at
FROM enriched
ON CONFLICT (order_id) DO UPDATE SET
    amount      = EXCLUDED.amount,
    order_tier   = EXCLUDED.order_tier,
    updated_at   = NOW();

```

Pattern 2: JSONB Flattening

```

-- ELT: Load raw JSON events, transform columns in-DB
INSERT INTO events_clean (event_id, user_id, event_type, properties, occurred_at)
SELECT
    (payload->>'id')::UUID,
    (payload->>'user_id')::INT,
    payload->>'event',
    payload->'properties',          -- keep nested JSONB
    (payload->>'timestamp')::TIMESTAMPTZ
FROM raw_events_json
WHERE loaded_at BETWEEN :batch_start AND :batch_end;

-- Further transform: extract nested properties
UPDATE events_clean
SET
    page_url   = properties->>'url',
    referrer   = properties->>'referrer',
    device     = properties->>'device_type'
WHERE page_url IS NULL
    AND event_type = 'page_view';

```

Pattern 3: Slowly Changing Dimension (SCD Type 2) in ELT

```

-- SCD2 in-DB transform
WITH incoming AS (
    SELECT customer_id, name, email, address, loaded_at
    FROM stg_customers
),
changed AS (
    SELECT i.*
    FROM incoming i
    LEFT JOIN dim_customers d
        ON d.customer_id = i.customer_id

```

```

        AND d.is_current = TRUE
    WHERE d.customer_id IS NULL          -- new customer
        OR d.email <> i.email           -- changed
        OR d.address <> i.address
    )
    -- Expire old rows
    UPDATE dim_customers
    SET is_current = FALSE,
        valid_to   = NOW()
    FROM changed
    WHERE dim_customers.customer_id = changed.customer_id
        AND dim_customers.is_current = TRUE;

    -- Insert new/changed rows
    INSERT INTO dim_customers (customer_id, name, email, address, valid_from, valid_to,
    is_current)
    SELECT customer_id, name, email, address, loaded_at, NULL, TRUE
    FROM changed;

```

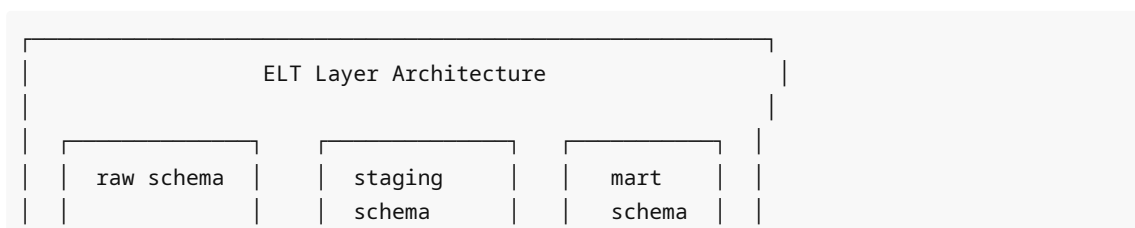
Pattern 4: Aggregate Mart Build

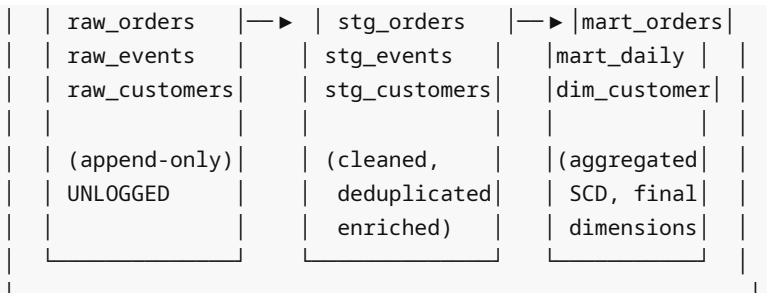
```

-- Build daily revenue mart from raw orders
INSERT INTO mart_daily_revenue
    (report_date, country_code, segment, total_orders, total_revenue,
    avg_order_value)
SELECT
    DATE_TRUNC('day', created_at)::DATE AS report_date,
    country_code,
    segment,
    COUNT(*)                          AS total_orders,
    SUM(amount)                       AS total_revenue,
    AVG(amount)                       AS avg_order_value
FROM mart_orders
WHERE created_at::DATE = CURRENT_DATE - 1 -- yesterday only
GROUP BY 1, 2, 3
ON CONFLICT (report_date, country_code, segment)
DO UPDATE SET
    total_orders    = EXCLUDED.total_orders,
    total_revenue   = EXCLUDED.total_revenue,
    avg_order_value = EXCLUDED.avg_order_value;

```

Staging Layer Design





```
-- Schema setup for medallion-style ELT
CREATE SCHEMA raw;
CREATE SCHEMA staging;
CREATE SCHEMA mart;

-- Raw layer: append-only, minimal constraints
CREATE UNLOGGED TABLE raw.orders (
  raw_id      BIGSERIAL PRIMARY KEY,
  payload     JSONB      NOT NULL,
  source      TEXT,
  loaded_at   TIMESTAMPTZ DEFAULT NOW(),
  batch_id    UUID       DEFAULT gen_random_uuid()
);

-- Staging layer: typed, deduplicated
CREATE TABLE staging.orders (
  order_id    BIGINT      PRIMARY KEY,
  customer_id INT,
  amount      NUMERIC(12,2),
  status      TEXT,
  created_at  TIMESTAMPTZ,
  stg_loaded  TIMESTAMPTZ DEFAULT NOW()
);

-- Mart layer: final, indexed, partitioned
CREATE TABLE mart.orders (
  order_id    BIGINT      PRIMARY KEY,
  customer_id INT          NOT NULL,
  amount      NUMERIC(12,2) NOT NULL,
  order_tier  TEXT,
  report_date DATE,
  created_at  TIMESTAMPTZ
) PARTITION BY RANGE (report_date);
```

Transform Layer Patterns

Idempotent Transform Function

```

CREATE OR REPLACE PROCEDURE elt_transform_orders(p_batch_date DATE)
LANGUAGE plpgsql AS $$
BEGIN
    -- Delete existing data for this batch date (idempotent)
    DELETE FROM staging.orders
    WHERE DATE(created_at) = p_batch_date;

    -- Re-insert from raw
    INSERT INTO staging.orders (order_id, customer_id, amount, status, created_at)
    SELECT
        (payload->>'order_id')::BIGINT,
        (payload->>'customer_id')::INT,
        (payload->>'amount')::NUMERIC(12,2),
        payload->>'status',
        (payload->>'created_at')::TIMESTAMPTZ
    FROM raw.orders
    WHERE loaded_at::DATE = p_batch_date
        AND payload->>'order_id' IS NOT NULL;

    RAISE NOTICE 'Transformed % rows for %', ROW_COUNT, p_batch_date;
END;
$$;

-- Call idempotently for any date
CALL elt_transform_orders('2024-01-15');
CALL elt_transform_orders('2024-01-15'); -- safe to re-run

```

Audit / Lineage Tracking

```

CREATE TABLE elt_audit_log (
    log_id          BIGSERIAL PRIMARY KEY,
    pipeline        TEXT      NOT NULL,
    batch_date      DATE,
    step            TEXT,
    rows_in         INT,
    rows_out        INT,
    rows_rejected   INT,
    started_at      TIMESTAMPTZ DEFAULT NOW(),
    finished_at     TIMESTAMPTZ,
    status          TEXT      DEFAULT 'running',
    error_detail    TEXT
);

-- Log within a procedure
INSERT INTO elt_audit_log (pipeline, batch_date, step, rows_in, rows_out, status)
VALUES ('orders_elt', CURRENT_DATE, 'raw_to_staging', v_rows_in, v_rows_out,
'success')
RETURNING log_id INTO v_log_id;

```

Orchestrating ELT with SQL

```
-- Master ELT orchestration procedure
CREATE OR REPLACE PROCEDURE run_elt_pipeline(p_date DATE DEFAULT CURRENT_DATE - 1)
LANGUAGE plpgsql AS $$
DECLARE
    v_start TIMESTAMPTZ := clock_timestamp();
BEGIN
    RAISE NOTICE 'ELT pipeline starting for %', p_date;

    -- Step 1: Raw → Staging
    CALL elt_transform_orders(p_date);
    CALL elt_transform_events(p_date);

    -- Step 2: Staging → Dim (SCD2 update)
    CALL elt_refresh_dim_customers();

    -- Step 3: Staging → Mart aggregates
    CALL elt_build_daily_revenue_mart(p_date);

    -- Step 4: Refresh materialized views
    REFRESH MATERIALIZED VIEW CONCURRENTLY mv_product_revenue;
    REFRESH MATERIALIZED VIEW CONCURRENTLY mv_customer_lifetime;

    RAISE NOTICE 'ELT pipeline complete in %', clock_timestamp() - v_start;

EXCEPTION WHEN OTHERS THEN
    INSERT INTO elt_audit_log(pipeline, batch_date, step, status, error_detail)
    VALUES ('master_elt', p_date, 'pipeline', 'failed', SQLERRM);
    RAISE;
END;
$;
```

Common Mistakes

Mistake	Issue	Solution
Transforming in raw layer	Raw data mutated, no recovery	Keep raw append-only
No idempotency	Duplicates on re-run	Use ON CONFLICT or delete-then-insert
Giant single-step transforms	Hard to debug, slow	Break into CTE stages
Skipping raw layer	No audit trail	Always store raw payload
Not partitioning raw tables	Raw layer grows indefinitely	Partition by loaded_at date

Forgetting ANALYZE	Bad query plans on staging tables	ANALYZE staging.orders after load
--------------------	-----------------------------------	-----------------------------------

Best Practices

1. **Medallion architecture:** raw → staging → mart — never skip layers.
2. **Raw is sacred** — append-only, never update or delete.
3. **Every transform is idempotent** — re-runnable without side effects.
4. **Capture lineage** — log every batch, row count, duration.
5. **Use schemas** (raw , staging , mart) to enforce layer boundaries.
6. **Partition mart tables** by date for query pruning.
7. **Keep transforms in SQL/plpgsql** — not Python — for portability and performance.

Performance Considerations

```
-- Vacuum and analyze after heavy ELT loads
VACUUM ANALYZE staging.orders;

-- Use parallel query for large transforms
SET max_parallel_workers_per_gather = 4;
SET parallel_tuple_cost = 0.1;

-- Monitor slow ELT queries
SELECT query, mean_exec_time, calls
FROM pg_stat_statements
WHERE query ILIKE '%mart_orders%'
ORDER BY mean_exec_time DESC
LIMIT 10;

-- Partial indexes for ELT filters
CREATE INDEX idx_raw_orders_unprocessed
ON raw.orders (loaded_at)
WHERE processed = FALSE;
```

Interview Questions & Answers

Q1: What is the fundamental difference between ETL and ELT?

In ETL, data is transformed before loading into the target. In ELT, raw data is loaded first, then transformed inside the target database using SQL. ELT leverages the database engine's power for transformation.

Q2: When would you prefer ELT over ETL?

ELT is preferred when the target database is powerful enough for transformations (PostgreSQL, BigQuery, Snowflake), when raw data preservation is required for re-processing, and when schemas evolve frequently (schema-on-read flexibility).

Q3: What is the medallion architecture?

A three-layer pattern: Bronze (raw, append-only), Silver (cleaned, deduplicated), Gold (aggregated, business-ready). In PostgreSQL, these map to `raw`, `staging`, and `mart` schemas.

Q4: How do you make an ELT transform idempotent?

Use `INSERT ... ON CONFLICT DO UPDATE`, or delete the target partition/date range before re-inserting. Store a `batch_id` to identify and remove previous runs.

Q5: How would you handle schema evolution in an ELT pipeline?

Store raw data as JSONB in the raw layer. Transformation SQL can then extract new fields without re-loading. Add `ALTER TABLE ... ADD COLUMN` with defaults to staging/mart layers.

Q6: What is the advantage of keeping raw data in JSONB vs typed columns?

JSONB preserves the original payload without schema commitment, allows replay when transform logic changes, and accommodates varying source schemas.

Q7: How do you debug a failed ELT transform in PostgreSQL?

Check `elt_audit_log` for the error step, run the failing CTE step in isolation with `EXPLAIN ANALYZE`, inspect `pg_stat_activity` for blocking queries, and query the staging table for malformed rows.

Q8: What PostgreSQL features make it suitable as an ELT engine?

CTEs, window functions, JSONB operators, FDWs, `plpgsql` stored procedures, parallel query execution, and materialized views collectively make PostgreSQL a full ELT platform.

Q9: How does dbt fit into an ELT architecture with PostgreSQL?

dbt is an ELT transformation framework. It compiles SQL models into `CREATE TABLE AS` or `INSERT ... SELECT` statements and runs them against PostgreSQL, adding lineage, testing, and documentation on top of raw SQL ELT.

Q10: Explain SCD Type 2 implementation in an ELT context.

SCD Type 2 preserves history by expiring old rows (`valid_to = NOW()`, `is_current = FALSE`) and inserting new rows for changed attributes. In ELT, this is done as a SQL `UPDATE + INSERT` pair inside the transform step, triggered when source data differs from the current dimension record.

Exercises & Solutions

Exercise 1: Build a three-layer ELT schema

```
CREATE SCHEMA raw; CREATE SCHEMA staging; CREATE SCHEMA mart;

CREATE UNLOGGED TABLE raw.products (
    raw_id BIGSERIAL, payload JSONB, loaded_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE staging.products (
    product_id INT PRIMARY KEY,
    name TEXT,
    price NUMERIC(10,2),
    category TEXT,
```

```

    stg_loaded    TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE mart.product_revenue (
    report_date    DATE,
    category       TEXT,
    total_revenue  NUMERIC,
    PRIMARY KEY (report_date, category)
);

```

Exercise 2: Idempotent CTE transform

```

-- Transform raw products into staging (idempotent)
INSERT INTO staging.products (product_id, name, price, category)
SELECT
    (payload->>'product_id')::INT,
    payload->>'name',
    (payload->>'price')::NUMERIC(10,2),
    payload->>'category'
FROM raw.products
WHERE loaded_at::DATE = CURRENT_DATE - 1
ON CONFLICT (product_id) DO UPDATE
    SET name = EXCLUDED.name, price = EXCLUDED.price, category = EXCLUDED.category;

```

Real-World Scenarios

Scenario: SaaS analytics platform A SaaS app logs user events to PostgreSQL as JSONB. ELT transforms them nightly:

- Raw: `raw.events` (JSONB, partitioned by day)
- Staging: `staging.events` (typed columns, deduplicated)
- Mart: `mart.feature_usage` (aggregated by feature, day, tenant) Transforms run as stored procedures called by Airflow PythonOperator.

Scenario: Multi-source product catalog Three vendors push CSV files. All loaded to `raw.vendor_products` with a `vendor_id` column. ELT merges them into a unified `mart.products` table using `ON CONFLICT` with a priority order (vendor A > B > C).

Cross-References

- `01_etl_with_postgresql.md` — COPY, staging, bulk loading
- `07_analytics_workloads.md` — window functions and aggregation strategies
- `09_postgresql_and_dbt.md` — dbt as an ELT orchestration layer
- `04_data_warehousing.md` — star schema and dimension design